

# Assignment 5: Autoencoders for Dimensionality Reduction

**Title (Use of ai is strictly prohibited- strict plagsim policy towards ai generated content)**

*Note – feel free to ask topics / issue and problems faced during assignments in doubt class*

Building Autoencoders for MNIST Image Compression and Denoising

## **Problem Statement**

Design and implement various autoencoder architectures (vanilla, denoising, variational) to learn compressed representations of MNIST digit images. You will explore dimensionality reduction, image denoising, and latent space visualization while understanding the principles of unsupervised representation learning.

## **Dataset**

### **MNIST Handwritten Digits**

- **Source:** [MNIST Database](#)
- **PyTorch:** torchvision.datasets.MNIST
- **Keras:** tensorflow.keras.datasets.mnist
- **Description:** 70,000 grayscale 28×28 images of digits (0-9), 60,000 training + 10,000 test

## **Detailed Instructions**

### **Part 1: Data Preparation (10%)**

1. Load MNIST dataset and visualize sample images
2. Normalize pixel values to [0, 1]
3. Flatten images to 784-dimensional vectors (for vanilla autoencoder)
4. Create noisy dataset for denoising autoencoder:
  - Add Gaussian noise: noise\_factor = 0.5
  - Clip values to [0, 1]
5. Visualize original vs. noisy images

## **Part 2: Vanilla Autoencoder (25%)**

**Task 2.1: Architecture Design** Implement a fully connected autoencoder:

Encoder:

Input(784) → Dense(512, ReLU) → Dense(256, ReLU) → Dense(128, ReLU) → Dense(32)

Decoder:

Dense(32) → Dense(128, ReLU) → Dense(256, ReLU) → Dense(512, ReLU) → Dense(784, Sigmoid)

### **Task 2.2: Training**

1. Loss function: Mean Squared Error (MSE) or Binary Cross-Entropy
2. Optimizer: Adam with lr=0.001
3. Batch size: 256
4. Epochs: 50
5. Track reconstruction loss

### **Task 2.3: Evaluation**

1. Visualize original vs. reconstructed images (grid of 10×10)
2. Calculate reconstruction error per digit class
3. Visualize 2D/3D latent space using t-SNE or PCA on 32-dim encodings
4. Color-code latent space by digit class

## **Part 3: Convolutional Autoencoder (25%)**

**Task 3.1: Architecture** Implement a convolutional autoencoder:

Encoder:

Conv2D(32, 3×3, stride=2, padding=1) → ReLU →  
Conv2D(64, 3×3, stride=2, padding=1) → ReLU →  
Conv2D(128, 3×3, stride=2, padding=1) → ReLU →  
Flatten → Dense(16)

Decoder:

Dense(128) → Reshape(128, 4, 4) →  
ConvTranspose2D(64, 3×3, stride=2, padding=1) → ReLU →  
ConvTranspose2D(32, 3×3, stride=2, padding=1) → ReLU →  
ConvTranspose2D(1, 3×3, stride=2, padding=1) → Sigmoid

### **Task 3.2: Training & Evaluation**

1. Train with same settings as vanilla autoencoder
2. Compare reconstruction quality with vanilla version
3. Visualize learned convolutional filters
4. Analyze latent space (now 16-dim)

### **Part 4: Denoising Autoencoder (20%)**

#### **Task 4.1: Implementation**

1. Use the convolutional autoencoder architecture
2. Train on noisy images, reconstruct clean images
3. Use both Gaussian noise and salt-and-pepper noise

#### **Task 4.2: Evaluation**

1. Calculate PSNR (Peak Signal-to-Noise Ratio) improvement
2. Calculate SSIM (Structural Similarity Index)
3. Visualize denoising results: noisy → denoised → original
4. Test on different noise levels (0.3, 0.5, 0.7)

### **Part 5: Variational Autoencoder (VAE) - Optional Advanced (20%)**

#### **Task 5.1: VAE Architecture**

1. Implement encoder that outputs mean ( $\mu$ ) and log-variance ( $\log \sigma^2$ )
2. Implement reparameterization trick:  $z = \mu + \sigma \odot \epsilon$
3. Implement KL divergence regularization

#### **Task 5.2: Loss Function**

- Total loss = Reconstruction Loss +  $\beta \times$  KL Divergence
- Experiment with  $\beta$  values (0.5, 1.0, 2.0)

#### **Task 5.3: Latent Space Exploration**

1. Visualize latent space distribution
2. Generate new digits by sampling from latent space
3. Perform latent space interpolation between two digits
4. Create a "digit manifold" by systematically varying latent dimensions

## **Expected Outputs**

### **1. Code Deliverables**

- Data preprocessing script
- Three separate model files (vanilla, convolutional, denoising)
- Training scripts for each model
- Evaluation and visualization notebook
- VAE implementation (if attempted)

### **2. Model Files**

- Trained weights for all autoencoder variants
- Training history (loss curves)

### **3. Visualizations (minimum 15)**

- Original vs. noisy images
- Reconstruction comparison for each model (10×10 grid)
- Reconstruction error heatmap per digit class
- Training loss curves for all models
- t-SNE/PCA visualization of latent space (colored by digit)
- Learned convolutional filters (first layer)
- Denoising results at different noise levels
- PSNR/SSIM comparison charts
- Latent space interpolation (if VAE)
- Generated samples from VAE (if implemented)
- Feature map visualizations from encoder layers
- Compression ratio vs. reconstruction quality trade-off

### **4. Performance Report (3-4 pages)**

- Architecture design rationale for each autoencoder type
- Comparison of vanilla vs. convolutional autoencoder
- Analysis of latent space structure
- Denoising performance metrics
- Discussion of compression trade-offs
- VAE analysis (if implemented)

5. **Comparative Analysis Table**

| Model     | Latent Dim | Reconstruction Loss | Training Time | Best Use Case |
|-----------|------------|---------------------|---------------|---------------|
| Vanilla   | 32         | X.XXX               | X min         | Dense data    |
| Conv      | 16         | X.XXX               | Y min         | Image data    |
| Denoising | 16         | X.XXX               | Y min         | Noise removal |
| VAE       | 20         | X.XXX               | Z min         | Generation    |

**Findings/Observations to Document**

- 1. How does latent dimension size affect reconstruction quality?
- 2. Why does convolutional autoencoder outperform vanilla for images?
- 3. What noise level can the denoising autoencoder effectively handle?
- 4. How is the latent space organized? Are similar digits clustered together?
- 5. What happens when you reduce latent dimensions too much?
- 6. How does VAE's latent space differ from deterministic autoencoders?
- 7. Can the autoencoder generalize to handwritten digits not in MNIST?

**Evaluation Rubric (100 points)**

| Criteria               | Points | Description                                                              |
|------------------------|--------|--------------------------------------------------------------------------|
| Code Implementation    | 35     | All three autoencoders correctly implemented, proper training loops      |
| Reconstruction Quality | 20     | Clear, recognizable reconstructions, MSE < 0.01 for vanilla/conv         |
| Denoising Performance  | 15     | Effective noise removal, PSNR improvement > 5dB                          |
| Visualizations         | 15     | All required plots, latent space analysis, clear comparisons             |
| Analysis & Report      | 10     | Insightful comparison, architectural understanding, trade-off discussion |
| Code Quality           | 5      | Clean, modular, well-documented code                                     |

### **Bonus Points (10)**

- Implement VAE with quality results (+5)
- Sparse autoencoder with L1 regularization (+2)
- Apply autoencoder to a different dataset (Fashion-MNIST) (+2)
- Interactive latent space explorer (Plotly) (+1)

### **Target Metrics**

- **Reconstruction MSE:**  $<0.015$  (vanilla),  $<0.010$  (convolutional)
- **Denoising PSNR:**  $>20\text{dB}$  improvement
- **VAE Generation:** Recognizable digits when sampling from  $N(0,1)$

\*\*\*\*\*